

Lab 8

ส่วนที่ 1: หลักการออกแบบ

1. SOLID Principles ที่ใช้และเกี่ยวข้องกับ Entity อย่างไร?

ตอบ S (Single Responsibility Principle)

- Product -> เก็บข้อมูลหลักของสินค้า
- ProductDetail -> เก็บข้อมูลเสริม
- Review -> เก็บข้อมูลรีวิว

O (Open/Closed Principle)

- ใน Strategy Pattern เช่น กรณีที่จะเพิ่มส่วนลดแบบใหม่ ก็แค่ Implements คลาส DiscountStrategy โดยไม่ต้องแก้ไขโค้ดเดิม

D (Dependency Inversion Principle)

- Product.getDiscountedPrice() และ DiscountContext ฟังก์ชัน interface DiscountStrategy ไม่ใช่ implementation ตรงๆ เช่นเดียวกับ ProductService ที่พึ่งพา ProductRepository

2. ความแตกต่างของ 1:1 กับ 1:N ใช้ในกรณีไหนควรใช้แบบไหน

ตอบ 1:1 : Product <-> ProductDetail – 1 สินค้ามีข้อมูลเสริมได้ 1 ชุด

Annotation: @OneToOne

เมื่อไหร่ควรใช้: ข้อมูลที่เป็นส่วนขยายของ Entity หลัก ไม่อยากให้ตารางหลักมีหลายคอลัมน์เยอะเกินไป

1:N : Product <-> Review – 1 สินค้ามีรีวิวได้หลายรายการ

Annotation: @OneToMany / @ManyToOne

เมื่อไหร่ควรใช้: ข้อมูลที่เกิดซ้ำได้ไม่จำกัดจำนวนต่อ 1 Record

3. Strategy Pattern สำหรับคำนวณส่วนลด

ตอบ ประกอบด้วย 3 ส่วน คือ

1. Interface – DiscountStrategy กำหนด Contract กลางว่าทุกกลยุทธ์ต้องมี method `apply(double price)`
2. Concrete Strategies – NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy แต่ละตัว implement logic ของตัวเอง
3. Context – DiscountContext ทำหน้าที่เลือกใช้ Strategy ตัวไหนตามค่า `discountType` ที่ส่งเข้ามา แล้วเลือกใช้ `calculate()`

4. Execution Flow ตั้งแต่ HTTP Request -> DB

ตอบ ยกตัวอย่างกรณีเข้าหน้า GET /products:

1. Browser ส่ง GET /products เข้ามาที่ Tomcat
2. DispatcherServlet รับ request แล้วหา controller ที่ map กับ path นี้
3. ProductController.list() ถูกเรียก
4. Controller เรียก productService.findAll()
5. ProductService เรียกต่อไปที่ ProductRepository.findAll() ซึ่งเป็น interface ของ Spring Data JPA
6. Spring Data JPA สร้าง implementation ให้อัตโนมัติ แล้วส่งต่อให้ Hibernate แปลงเป็น SQL query
7. Hibernate ยิง query ไปที่ PostgreSQL database ผ่าน JDBC driver
8. ผลลัพธ์จาก DB ถูกแปลงเป็น List<Product>
9. Controller เอา list นี้ใส่ใน model
10. Controller return ชื่อ view

11. Thymeleaf ViewResolver หาไฟล์ templates/products/list.html มา render โดยดึงข้อมูลจาก Model ไปแสดงผล
12. HTML ที่ render เสร็จแล้วถูกส่งกลับไป browser เป็น response

ส่วนที่ 2: Code + คำอธิบาย

- Entity: Product

```
package com.example.demo.model;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OneToOne;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)
    private ProductDetail detail;

    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    public Product() {
    }
}
```

```
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public String getBrand() {
    return brand;
}

public void setBrand(String brand) {
    this.brand = brand;
}
}
```

```

public Integer getStock() {
    return stock;
}

public void setStock(Integer stock) {
    this.stock = stock;
}

public Double getPrice() {
    return price;
}

public void setPrice(Double price) {
    this.price = price;
}

public String getDiscountType() {
    return discountType;
}

public void setDiscountType(String discountType) {
    this.discountType = discountType;
}

public ProductDetail getDetail() {
    return detail;
}

public void setDetail(ProductDetail detail) {
    this.detail = detail;
}

```

```

public List<Review> getReviews() {
    return reviews;
}

public void setReviews(List<Review> reviews) {
    this.reviews = reviews;
}

public Double getDiscountedPrice() {
    return new com.example.demo.strategy.DiscountContext(discountType).calculate(price);
}
}

```

- @Entity - บอก JPA/Hibernate ว่า class นี้แมปกับตารางในฐานข้อมูล
- @Id - กำหนดว่า field id คือ primary key
- @GeneratedValue(strategy=GenerationType.IDENTITY) - ให้ฐานข้อมูลเป็นคน Generate ค่า id ให้อัตโนมัติ
- @OneToOne(cascade=CascadeType.ALL, fetch=FetchType.Lazy, orphanRemoval=true) -
 1. cascade= ALL -> เวลา save/delete Product ให้ทำ operation เดียวกันกับ ProductDetail ที่ผูกอยู่ด้วยอัตโนมัติ
 2. fetch = LAZY -> ไม่ต้องดึง ProductDetail ออกมาจาก DB ทันทีตอนโหลด Product จะโหลดก็ต่อเมื่อถูกเรียกใช้จริง
 3. orphanRemoval = true -> ถ้า Product ไม่ผูกกับ ProductDetail เดิมแล้ว ให้ลบ ProductDetail ตัวเก่าทิ้งจาก DB ด้วย ไม่ปล่อยให้ข้อมูลร้าง

- @OneToMany(mappedBy="product", cascade=CascadeType.ALL, orphanRemoval=true)
 1. mappedBy = "product" -> บอกว่าฝั่งที่เป็นเจ้าของความสัมพันธ์จริง คือ field product ใน class Review ตารางฐานข้อมูลจะมี foreign key อยู่ที่ review ไม่ใช่ product
 2. cascade = ALL + orphanRemoval = True -> ทำงานเหมือน 1:1 ด้านบน แต่ใช้กับ list ของ review แทน

- Entity: ProductDetail

```
package com.example.demo.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class ProductDetail {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String description;
    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;

    public ProductDetail() {
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getWarranty() {
        return warranty;
    }

    public void setWarranty(String warranty) {
        this.warranty = warranty;
    }

    public Double getWeight() {
        return weight;
    }

    public void setWeight(Double weight) {
        this.weight = weight;
    }

    public String getDimensions() {
        return dimensions;
    }

    public void setDimensions(String dimensions) {
        this.dimensions = dimensions;
    }
}
```

```

public String getManufacturedCountry() {
    return manufacturedCountry;
}

public void setManufacturedCountry(String manufacturedCountry) {
    this.manufacturedCountry = manufacturedCountry;
}

public Product getProduct() {
    return product;
}

public void setProduct(Product product) {
    this.product = product;
}
}

```

- OneToOne(mappedBy = "detail") – เป็นฝั่ง inverse side ตัว mappedBy = "detail" นี้กลับไปหา field detail ใน Product เพื่อบอกว่าความสัมพันธ์จริงถูกจัดการที่ฝั่ง Product เท่านั้น field นี้มีไว้เพื่อให้ navigate กลับไปหา Product เจ้าของได้เฉยๆ ไม่ได้สร้าง foreign key ซ้ำซ้อน

- Entity: Review

```

package com.example.demo.model;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import java.time.LocalDate;

@Entity
public class Review {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String reviewer;
    private Integer rating;
    private String comment;
    private LocalDate reviewDate = LocalDate.now();

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id")
    private Product product;

    public Review() {
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
}

```

```

public String getReviewer() {
    return reviewer;
}

public void setReviewer(String reviewer) {
    this.reviewer = reviewer;
}

public Integer getRating() {
    return rating;
}

public void setRating(Integer rating) {
    this.rating = rating;
}

public String getComment() {
    return comment;
}

public void setComment(String comment) {
    this.comment = comment;
}

public LocalDate getReviewDate() {
    return reviewDate;
}

public void setReviewDate(LocalDate reviewDate) {
    this.reviewDate = reviewDate;
}

public Product getProduct() {
    return product;
}

public void setProduct(Product product) {
    this.product = product;
}
}

```

- @ManyToOne(fetch = FetchType.LAZY) - ฝั่งนี้คือ owing side ของความสัมพันธ์ 1:N fetch = LAZY ทำให้ไม่ต้องโหลดข้อมูล Product เต็มๆ ทุกครั้งที่ดึง Review มา
- @JoinColumn(name = "product_id") - กำหนดชื่อ foreign key column ในตาราง view ให้ชัดเจนว่าชื่อ product_id

- Service: ProductService

```
package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.model.Review;
import com.example.demo.repository.ProductRepository;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Transactional(readOnly = true)
    public List<Product> findAll() {
        return productRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Product findById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("ไม่พบสินค้า id: " + id));
    }

    @Transactional
    public Product save(Product product) {
        prepareRelationships(product);
        return productRepository.save(product);
    }

    @Transactional
    public Product update(Long id, Product submitted) {
        Product existing = findById(id);
        existing.setName(submitted.getName());
        existing.setCategory(submitted.getCategory());
        existing.setBrand(submitted.getBrand());
        existing.setStock(submitted.getStock());
        existing.setPrice(submitted.getPrice());
        existing.setDiscountType(submitted.getDiscountType());
        if (submitted.getDetail() != null) {
            submitted.getDetail().setProduct(existing);
            existing.setDetail(submitted.getDetail());
        }
        return productRepository.save(existing);
    }

    @Transactional
    public void delete(Long id) {
        productRepository.deleteById(id);
    }

    private void prepareRelationships(Product product) {
        if (product.getDetail() != null) {
            product.getDetail().setProduct(product);
        }
        if (product.getReviews() != null) {
            product.getReviews().removeIf(this::isBlankReview);
            product.getReviews().forEach(review -> review.setProduct(product));
        }
    }

    private boolean isBlankReview(Review review) {
        return review == null || (review.getReviewer() == null && review.getComment() == null);
    }
}
```

- @Service - บอก Spring ว่า class นี้เป็น business logic layer ให้สร้างเป็น bean แล้วจัดการโดย spring container

- Constructor Injection คือการรับ dependency ผ่าน constructor แทนการใช้ @Autowired บน field ตรงๆ
- ข้อดีของ Constructor Injection เทียบกับ Field Injection
 1. ทำให้ ProductRepository เป็น final ได้ -> immutable, เปลี่ยนค่าที่หลังไม่ได้
 2. บังคับว่าต้องมี dependency ครบก่อนจะสร้าง object ได้
 3. เขียน unit test ง่ายกว่าเพราะสามารถ new ProductService(mockRepository) ได้ตรงๆ โดยไม่ต้องพึ่ง Spring Container
- Controller: ProductController

```
package com.example.demo.controller;

import com.example.demo.model.Product;
import com.example.demo.model.ProductDetail;
import com.example.demo.model.Review;
import com.example.demo.service.ProductService;
import java.util.ArrayList;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String list(Model model) {
        model.addAttribute("products", productService.findAll());
        return "products/list";
    }

    @GetMapping("/add")
    public String addForm(Model model) {
        Product product = new Product();
        product.setDetail(new ProductDetail());
        product.setReviews(new ArrayList<>());
        product.getReviews().add(new Review());
        model.addAttribute("product", product);
        return "products/add";
    }
}
```

```

@PostMapping("/save")
public String save(Product product, RedirectAttributes redirectAttributes) {
    productService.save(product);
    redirectAttributes.addFlashAttribute("message", "บันทึกสินค้าเรียบร้อยแล้ว");
    return "redirect:/products";
}

@GetMapping("/edit/{id}")
public String editForm(@PathVariable Long id, Model model) {
    model.addAttribute("product", productService.findById(id));
    return "products/edit";
}

@PostMapping("/update/{id}")
public String update(@PathVariable Long id, Product product,
    RedirectAttributes redirectAttributes) {
    productService.update(id, product);
    redirectAttributes.addFlashAttribute("message", "แก้ไขสินค้าเรียบร้อยแล้ว");
    return "redirect:/products";
}

@GetMapping("/delete/{id}")
public String deleteForm(@PathVariable Long id, Model model) {
    model.addAttribute("product", productService.findById(id));
    return "products/delete";
}

@PostMapping("/delete/{id}")
public String delete(@PathVariable Long id, RedirectAttributes redirectAttributes) {
    productService.delete(id);
    redirectAttributes.addFlashAttribute("message", "ลบสินค้าเรียบร้อยแล้ว");
    return "redirect:/products";
}
}

```

- @Controller - ต่างจาก @RestController ต่างตรงที่ method ที่ return String จะถูกตีความว่าเป็นชื่อ view ไม่ใช่ return เป็น response body ตรงๆ แบบ RestAPI
- @RequestMapping("/products") บน class - ตั้ง prefix ของ url ทุก method ในนี้ให้ขึ้นต้นด้วย /products เช่น @GetMapping("/add") จริงๆ คือ /products/add
- Constructor Injection เหมือนกับ ProductService – ProductController รับ ProductService ผ่าน Constructor ทำให้ Controller layer พึ่งพา Service Layer ผ่าน Abstraction ไม่ต้องไปยุ่งกับ ProductRepository โดยตรง ทำให้แยกความรับผิดชอบระหว่างชั้น presentation กับ business logic ได้ชัดเจน